DES PUNE UNIVERSITY
School of Engineering and Technology
Computer Engineering and Technology
Program: B.Tech. Computer Science and Engineering

Academic Year: 2025-26	Year: Second Year	Term: I
Roll No.: 1012411011	Name: Riddhee Kulkarni	
Subject: Java		
Assignment No.: 12	Title: RMI Programming	
Date: 12/10/2025		

Aim:

Develop a simple client-server application using RMI Programming.

Theory:

RMI (Remote Method Invocation) is a Java technology that allows a program to call methods on objects located on another computer or JVM over a network. It makes distributed programming easier because the programmer can call remote methods just like normal local methods.

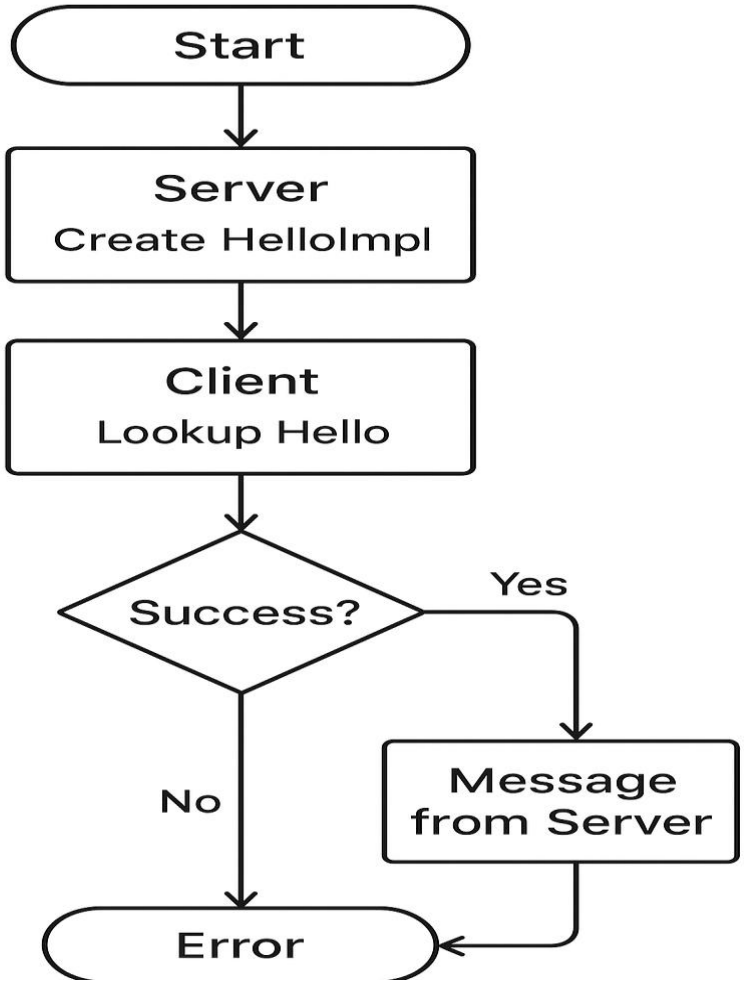
In an RMI client-server system, the server creates and hosts remote objects that provide services, while the client looks up these objects and calls their methods to get results. The communication is handled automatically by RMI, so programmers don't need to deal with low-level networking.

An RMI application has four main parts:

1. Remote Interface – defines the methods that the client can call remotely.
2. Remote Object Implementation – provides the actual code for these methods.
3. Server – creates the remote object, starts the RMI registry, and binds the object with a name.
4. Client – finds the remote object in the registry and calls its methods.

RMI is useful for building distributed applications because it allows objects on different machines to interact seamlessly while hiding network communication details.

Flowchart:



Code:

Hello.java

```
package Assignment12;
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote
{
    String sayHello() throws RemoteException;
}
```

HelloImpl.java

```java
package Assignment12;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class HelloImpl extends UnicastRemoteObject implements Hello
{
    protected HelloImpl() throws RemoteException
    {
        super();
    }

    public String sayHello() throws RemoteException
    {
        return "Hello from the RMI Server!";
    }
}
```

Server.java

```java
package Assignment12;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class Server {
    public static void main(String[] args)
    {
        try
        {
            HelloImpl obj = new HelloImpl();
            LocateRegistry.createRegistry(1099);
            Naming.rebind("Hello", obj);
            System.out.println("Server is ready...");
        }
        catch (Exception e)
        {
            System.out.println("Server Error: " + e);
        }
    }
}
```

Client.java

```java
package Assignment12;
import java.rmi.Naming;

public class Client
{
    public static void main(String[] args)
    {
        try
        {
            Hello obj = (Hello) Naming.lookup("rmi://localhost/Hello");
            System.out.println("Message from Server: " + obj.sayHello());
        }
        catch (Exception e)
        {
            System.out.println("Client Error: " + e);
        }
    }
}
```

Output:

Server:

```
Server is ready...
```

Client:

```
Message from Server: Hello from the RMI Server!
```

Conclusion:

A simple client-server application using RMI Programming is developed.